

CRRSS CLI User Guide

Complete Standalone CLI Tool (Phase 2 - Stage 4)

Version: 1.0.0

Date: 2025-10-12

Author: BDI Development Team

Table of Contents

1. [Introduction](#)
 2. [Installation](#)
 3. [Quick Start](#)
 4. [Commands Overview](#)
 5. [Personality Profiles](#)
 6. [Configuration Management](#)
 7. [Pre-Generation Consultation](#)
 8. [Code Validation](#)
 9. [Bug Pattern Lookup](#)
 10. [Interactive Mode](#)
 11. [Advanced Usage](#)
 12. [Examples](#)
 13. [Troubleshooting](#)
-

Introduction

CRRSS (Code Review, Reliability, and Static Safety System) is a comprehensive static analysis and code quality assurance tool designed for the Binary Decomposition Interface (BDI) project. It provides multiple personality profiles that analyze code from different perspectives, detecting bugs, security vulnerabilities, memory safety issues, and type safety problems.

Key Features

- **Multiple Personality Profiles:** MSM, STP, TDT, RERS, BPME
 - **Pre-generation Consultation:** Automatic project analysis and configuration suggestions
 - **Comprehensive Validation:** Multi-profile code analysis
 - **Bug Pattern Lookup:** Database of common bug patterns with examples
 - **Configuration Management:** Flexible .crrsrc configuration files
 - **Interactive Mode:** Shell-like interface for exploration
 - **System-wide Installation:** Install once, use anywhere
-

Installation

Building from Source

```
cd tools/crrss
make clean
make all
```

System-wide Installation

```
sudo make install
# Default installation: /usr/local/bin/crrss
```

Custom Installation Path

```
make install INSTALL_PREFIX=/opt/crrss
```

Uninstallation

```
sudo make uninstall
```

Quick Start

1. Initialize Configuration

```
crrss configure --init
```

This creates a `.crrssrc` file in the current directory with default settings.

2. Get Project Consultation

```
crrss consult -d /path/to/your/project -o .crrssrc
```

CRRSS will analyze your project and suggest appropriate profiles and configuration.

3. Run Comprehensive Validation

```
crrss validate -d /path/to/code --use-all-profiles --report validation_report.txt
```

4. View Results

```
cat validation_report.txt
```

Commands Overview

Core Commands

Command	Description
query	Query bug predictions and risk assessments
stats	Display codebase statistics and system health
validate	Comprehensive validation with multiple profiles
consult	Pre-generation consultation and project analysis
configure	Configuration file management
profile	Personality profile selection and management
lookup	Bug pattern database lookup
interactive	Enter interactive mode

Personality Profile Commands

Command	Description
msm	Memory Safety Maniac - memory safety analysis
stp	Strict Typist Profile - type safety analysis
tdt	Test-Driven Timmy - test generation and coverage
rers	Runtime Error Replay System - error analysis

Utility Commands

Command	Description
help	Display help message
version	Display version information

Personality Profiles

1. MSM - Memory Safety Maniac

Purpose: Comprehensive memory safety analysis for C23 code.

Features:

- Memory leak detection
- Use-after-free detection
- Double-free detection
- NULL dereference detection
- Buffer overflow/underflow detection
- Missing NULL checks

Usage:

```
# Analyze a single file
crrss msm -f kernel/memory.c

# Analyze entire directory
crrss msm -d moduler_kernel/

# Generate report
crrss msm -d moduler_kernel/ --report msm_report.txt --format text
```

Configuration (.crrssrc):

```
[msm]
tracking_mode = detailed
detect_leaks = true
detect_use_after_free = true
detect_double_free = true
detect_null_deref = true
detect_buffer_overflow = true
max_tracked_allocations = 10000
```

2. STP - Strict Typist Profile

Purpose: Type safety validation and struct integrity analysis.

Features:

- Type mismatch detection
- Implicit conversion detection
- Signed/unsigned mixing
- Unsafe cast detection
- Struct alignment analysis
- Padding optimization suggestions
- Portability issue detection

Usage:

```
# Basic analysis
crrss stp -f kernel/types.c

# Strict mode (paranoid)
crrss stp -d moduler_kernel/ --strictness 3

# Generate report
crrss stp -d moduler_kernel/ --report stp_report.txt
```

Strictness Levels:

- **0 - Permissive:** Allow most conversions
- **1 - Moderate:** Warn on potentially unsafe operations (default)
- **2 - Strict:** Strict type checking
- **3 - Paranoid:** Maximum type safety enforcement

Configuration (.crrsrc):

```
[stp]
strictness_level = strict
check_type_safety = true
check_struct_alignment = true
check_type_casts = true
```

3. TDT - Test-Driven Timmy

Purpose: Automated test generation and code coverage analysis.

Status: Under development

Planned Features:

- Automatic test generation
- Code coverage analysis
- Test quality assessment
- Test template generation

4. RERS - Runtime Error Replay System

Purpose: Runtime error analysis and reproduction.

Features:

- Error replay and reproduction
- Active learning from errors
- Bug pattern detection
- Integration with other profiles

Usage:

```
# Basic usage
crrss rers --error-log /var/log/kernel_errors.log

# With replay target
crrss rers --error-log errors.log --replay-target target_function
```

5. BPME - Bug Prior Mapping Engine

Purpose: Historical bug pattern analysis and prediction.

Features:

- Bug pattern matching based on historical data (PRs #1-165)
- Risk assessment
- Priority assignment
- Confidence scoring

Usage:

BPME is automatically integrated with other commands (query, validate, etc.)

```
# Query by priority
crrss query --priority P0 --details

# Query by category
crrss query --category memory --max-results 50

# Query specific file
crrss query -f kernel/memory.c --details
```

Configuration Management

Initialize Configuration

```
crrss configure --init
```

Creates `.crrssrc` in the current directory.

Show Current Configuration

```
crrss configure --show
```

Validate Configuration

```
crrss configure --validate -f .crrssrc
```

Configuration File Format

Example `.crrssrc` :

```
# CRRSS Configuration File

[general]
version = 1.0.0
enable_strict_mode = true
max_issues = 1000

[profiles]
# Enable/disable personality profiles
msm = true # Memory Safety Maniac
stp = true # Strict Typist Profile
tdt = false # Test-Driven Timmy
rers = false # Runtime Error Replay System
bpme = true # Bug Prior Mapping Engine

[msm]
tracking_mode = detailed
detect_leaks = true
detect_use_after_free = true
detect_double_free = true
detect_null_deref = true
detect_buffer_overflow = true
max_tracked_allocations = 10000

[stp]
strictness_level = strict
check_type_safety = true
check_struct_alignment = true
check_type_casts = true

[bpme]
enable_pattern_matching = true
enable_ml_predictions = false
confidence_threshold = 0.5
max_predictions = 1000

[output]
report_format = text
report_directory = /tmp/crrss_reports
generate_html = false
generate_json = false
```

Pre-Generation Consultation

The consultation module analyzes your project and suggests appropriate profiles and configuration.

Basic Usage

```
crrss consult -d /path/to/project
```

Auto-detect Project Type

```
crrss consult -d /path/to/project --type kernel
```

Save Configuration

```
crrss consult -d /path/to/project -o .crrssrc
```

Example Output

```
=== CRRSS Pre-Generation Consultation ===

Analyzing project characteristics...
Project directory: /path/to/project

Detection Results:
  Project Type: Kernel/System
  Memory-Intensive: Yes

=== Recommended Profiles ===

For Kernel/System Projects:
  ☒ MSM (Memory Safety Maniac) - HIGH PRIORITY
    - Detects memory leaks, use-after-free, double-free
    - Essential for kernel-level memory safety

  ☒ STP (Strict Typist Profile) - RECOMMENDED
    - Type safety validation
    - Struct alignment and padding analysis

  ☒ BPME (Bug Prior Mapping Engine) - RECOMMENDED
    - Bug pattern detection based on historical data

=== Next Steps ===

1. Review the suggested configuration
2. Save configuration: crrss configure --init
3. Run validation: crrss validate -d . --use-all-profiles
4. Review results and adjust configuration as needed
```

Code Validation

The `validate` command provides comprehensive multi-profile analysis.

Basic Usage

```
# Validate single file
crrss validate -f kernel/memory.c

# Validate directory
crrss validate -d moduler_kernel/
```


Profile Selection

```
# Use specific profiles
crrss validate -d moduler_kernel/ --use-msm --use-stp

# Use all profiles
crrss validate -d moduler_kernel/ --use-all-profiles
```

Generate Report

```
crrss validate -d moduler_kernel/ \
  --use-all-profiles \
  --report validation_report.txt \
  --format text
```

Example Output

```
=== CRRSS Comprehensive Code Validation ===

Validation Target: moduler_kernel/
Type: Directory

Active Profiles:
☒ MSM (Memory Safety Maniac)
☒ STP (Strict Typist Profile)
☒ BPME (Bug Prior Mapping Engine)

=== Running MSM Analysis ===
Found 12 memory safety issues

Issue Breakdown:
  Use-After-Free:      2
  Double-Free:         0
  Memory Leaks:        5
  NULL Dereferences:   3
  Buffer Overflows:    2

=== Running STP Analysis ===
Found 8 type safety issues

Issue Breakdown:
  Type Mismatches:      2
  Implicit Conversions: 3
  Unsafe Casts:         2
  Struct Issues:        1

=== Validation Summary ===

Total Issues Found: 20
Status: ☒ FAILED - Multiple issues detected
```

Bug Pattern Lookup

The `lookup` command provides access to the bug pattern database.

List All Patterns

```
crrss lookup --list-all
```

Lookup Specific Pattern

```
crrss lookup --pattern MEMORY_LEAK
```

Example output:

Pattern Details: MEMORY_LEAK

PATTERN_MEMORY_LEAK

Priority: P1 (High)

Category: Memory

Risk: High

Description:

Memory allocation without corresponding deallocation, leading to gradual memory exhaustion.

Detection:

- Tracks malloc/calloc/realloc calls
- Verifies corresponding free calls
- Checks for all code paths

Example:

```
void* ptr = malloc(100);
if (error_condition) {
    return; // LEAK: ptr not freed
}
free(ptr);
```

Recommendation:

Ensure every allocation has a corresponding free on all code paths.
Consider using RAII-style patterns or cleanup handlers.

Query by Category

```
crrss lookup --category memory --details
```

Interactive Mode

Interactive mode provides a shell-like interface for exploration.

Start Interactive Mode

```
crrss interactive
```

or

```
crrss -i
```

Available Commands

```
crrss> help
Available commands:
  query      - Query bug patterns
  stats      - Show statistics
  msm        - Run MSM analysis
  stp        - Run STP analysis
  validate   - Comprehensive validation
  lookup     - Bug pattern lookup
  profiles   - List available profiles
  config     - Show configuration
  help       - Show this help
  exit     - Exit interactive mode
```

Example Session

```
crrss> profiles
=== CRRSS Personality Profiles ===
[Lists all available profiles]

crrss> config
[Shows current configuration]

crrss> stats
[Shows system statistics]

crrss> exit
Goodbye!
```

Advanced Usage

1. Pipeline Integration

```
# Check exit code
crrss validate -f file.c --use-msm
if [ $? -ne 0 ]; then
    echo "Validation failed!"
    exit 1
fi
```

2. Batch Processing

```
# Process multiple files
for file in $(find . -name "*.c"); do
    echo "Analyzing $file..."
    crrss msm -f "$file" --report "${file}.report"
done
```

3. CI/CD Integration

```
# .gitlab-ci.yml or .github/workflows
script:
  - crrss configure --init
  - crrss validate -d . --use-all-profiles --report crrss_report.txt
  - if [ $? -ne 0 ]; then exit 1; fi
artifacts:
  paths:
    - crrss_report.txt
```

4. Custom Configuration per Directory

```
# Different configs for different modules
cd kernel/
crrss configure --init
# Edit .crrssrc for kernel-specific settings

cd ../userspace/
crrss configure --init
# Edit .crrssrc for userspace-specific settings
```

Examples

Example 1: New Project Setup

```
# Step 1: Get consultation
crrss consult -d my_project/ -o .crrssrc

# Step 2: Review and adjust configuration
vim .crrssrc

# Step 3: Run initial validation
crrss validate -d my_project/ --use-all-profiles --report initial_report.txt

# Step 4: Review report
less initial_report.txt
```

Example 2: Memory Safety Focus

```
# Configure for memory safety
cat > .crrssrc << EOF
[profiles]
msm = true
bpme = true

[msm]
tracking_mode = detailed
detect_leaks = true
detect_use_after_free = true
detect_double_free = true
EOF

# Run MSM analysis
crrss msm -d kernel/ --report msm_report.html --format html
```

Example 3: Type Safety Analysis

```
# Run strict type checking
crrss stp -d moduler_kernel/ \
  --strictness 3 \
  --report stp_report.txt \
  --format text

# Review struct alignment issues
grep "STRUCT_PADDING" stp_report.txt
```

Example 4: Bug Pattern Research

```
# List all memory-related patterns
crrss lookup --list-all | grep "Memory Safety"

# Get details on specific patterns
crrss lookup --pattern MEMORY_LEAK > memory_leak_info.txt
crrss lookup --pattern USE_AFTER_FREE > use_after_free_info.txt
crrss lookup --pattern DOUBLE_FREE > double_free_info.txt
```

Troubleshooting

Issue: “Error: Failed to initialize CRRSS”

Solution:

- Check that you have proper permissions
- Ensure configuration file is valid
- Try running with `--init` to create default config

Issue: “Configuration file not found”

Solution:

```
# Create default configuration
crrss configure --init
```

Issue: “Too many issues detected”

Solution:

```
# Increase max issues limit
crrss msm -f file.c --max-issues 5000
```

or edit `.crrssrc` :

```
[general]
max_issues = 5000
```

Issue: “Profile not enabled”

Solution:

Check `.crrssrc` and enable the profile:

```
[profiles]
msm = true
stp = true
```

Issue: Build failures

Solution:

```
# Clean and rebuild
make clean
make all

# Check for tab vs space issues
python3 fix_makefile_tabs.py
```

Best Practices

1. Always start with consultation:

```
bash
crrss consult -d . -o .crrssrc
```

2. Use appropriate profiles for your project type:

- Kernel projects: MSM + STP + BPME
- User-space: MSM + TDT + BPME
- Libraries: MSM + STP + BPME

3. Adjust strictness levels based on needs:

- Development: Moderate (1)
- Pre-release: Strict (2)
- Critical systems: Paranoid (3)

4. Regularly run validation:

```
bash
# Daily or on every commit
crrss validate -d . --use-all-profiles
```

5. Keep configuration in version control:

```
bash
git add .crrssrc
git commit -m "Add CRRSS configuration"
```

6. Review reports systematically:

- Start with P0 (Critical) issues
- Then P1 (High) issues
- Address P2/P3 as time permits

Support and Contributing

Documentation

- Main README: `README.md`
- This User Guide: `CLI_USER_GUIDE.md`
- Integration Guide: `docs/INTEGRATION.md`
- Usage Guide: `docs/USAGE.md`

Reporting Issues

Please report issues to the BDI project repository with:

- CRRSS version (`crrss version`)
- Command that failed
- Error messages
- Configuration file (`.crrssrc`)

Contributing

We welcome contributions! See the main BDI contributing guidelines.

Appendix A: Command Reference

Complete Option List

```
crrss <command> [options]
```

Global Options:

```
-h, --help          Show help
-v, --version        Show version
```

Common Options:

```
-f, --file <path>    Analyze specific file
-d, --directory <path> Analyze directory
-r, --report <path>   Generate report
-F, --format <fmt>    Report format (text, json, html)
```

MSM Options:

```
--max-issues <n>      Maximum issues to report
```

STP Options:

```
-s, --strictness <0-3> Strictness level
```

Validate Options:

```
--use-msm            Use MSM profile
--use-stp            Use STP profile
--use-tdt            Use TDT profile
--use-rers           Use RERS profile
--use-all-profiles   Use all profiles
```

Consult Options:

```
-t, --type <type>    Project type
-o, --output <path>   Save configuration
```

Configure Options:

```
--init              Initialize configuration
--show              Show configuration
--validate          Validate configuration
```

Profile Options:

```
--list              List profiles
--select <profile>  Select profile
--info <profile>    Show profile info
```

Lookup Options:

```
--pattern <name>    Lookup pattern
--list-all          List all patterns
--details            Show details
```

Appendix B: Exit Codes

Code	Meaning
0	Success
1	General error
2	Invalid parameters
3	Not initialized
4	File access error
5	Validation failed

Appendix C: File Formats

Report Formats

- **text**: Human-readable plain text
- **json**: Machine-readable JSON
- **html**: Web-viewable HTML report

Configuration Format

INI-style configuration with sections:

- `[general]` : Global settings
 - `[profiles]` : Profile enable/disable
 - `[msm]` , `[stp]` , `[bpme]` : Profile-specific settings
 - `[output]` : Output settings
-

End of User Guide

For the latest updates and documentation, visit the BDI project repository.